

VEMTAP FINANCIAL DASHBOARD UI DESIGN

1. OVERVIEW

This is an internal Financial Command Center for Vemtap. It is designed for founders, finance admins, and selected managers only.

Goal: Provide real-time visibility into revenue, costs, agents, and growth.

2. MAIN DASHBOARD (HOME SCREEN)

Top Summary Cards (First Row)

- Total MRR (Monthly Recurring Revenue)
- Total Active Businesses
- Net Profit (This Month)
- Total SMS Revenue

Second Row

- Vemtap Revenue (Subscriptions)
- QRThrive Revenue
- Total Commissions Paid
- Cash Balance (Manual input or integrated later)

Third Row (Quick Insights)

- Best Performing Agent
 - Worst Performing Agent
 - Conversion Rate (QRThrive → Vemtap)
 - Churn Rate
-

3. REVENUE ANALYTICS PAGE

Charts

- Revenue Growth (Monthly line chart)
- Vemtap vs QRThrive (bar chart)
- Revenue by Plan (pie chart)

Table

Columns:

- Date
- Revenue Type (Subscription / SMS / Enterprise)
- Amount
- Platform (Vemtap / QRThrive)

Filters:

- Date range
 - Platform
 - Revenue type
-

4. AGENT PERFORMANCE DASHBOARD

Summary Cards

- Total Agents
- Active Agents
- Total Revenue Generated by Agents
- Total Commission Paid

Table

Columns:

- Agent Name
- Businesses Acquired
- Total Revenue Generated
- Commission Earned
- Conversion Rate
- Status (Active / Inactive)

Drill-down (Click Agent)

- Monthly performance chart
 - Businesses list
 - Earnings breakdown
-

5. SUBSCRIPTION & BUSINESS TRACKER

Summary

- Total Businesses
- Active vs Inactive
- Free vs Paid Ratio

Plan Distribution Chart

- Silver
- Gold
- Platinum
- Enterprise

Table

Columns:

- Business Name
 - Plan
 - Monthly Payment
 - Join Date
 - Last Payment Date
 - Status
-

6. SMS REVENUE TRACKER

Summary Cards

- Total SMS Sent
- Total SMS Revenue
- Total SMS Profit

Chart

- SMS Usage Over Time

Table

Columns:

- Business Name

- SMS Sent
 - Cost
 - Revenue
 - Profit
-

7. QRTHRIVE FUNNEL DASHBOARD

Funnel View

- Total QR Scans
- Leads Captured
- QRThrive Users
- Converted to Vemtap

Metrics

- Conversion Rate (%)
 - Revenue Generated from Converted Users
-

8. EXPENSE & COST TRACKER

Fixed Costs

- Salaries
- Hosting (Vemtap)
- Hosting (QRThrive)
- Tools
- Miscellaneous

Variable Costs

- Commissions
- Payment processing fees
- SMS cost

Monthly Expense Summary

Chart showing total expenses vs revenue

9. PROFIT & LOSS VIEW

Monthly P&L Summary

- Total Revenue
- Total Cost
- Net Profit

Chart

- Profit Trend Over Time
-

10. CASH FLOW DASHBOARD

Inflows

- Subscription payments
- SMS revenue

Outflows

- Salaries
- Commissions
- Infrastructure

Net Cash Position

- Opening Balance
 - Closing Balance
-

11. ALERTS & INSIGHTS PANEL

System-generated insights such as:

- "Revenue dropped by 15% this week"
 - "Agent John generated 40% of total revenue"
 - "Churn increased above 10%"
 - "You may run out of cash in 3 months"
-

12. NAVIGATION STRUCTURE (SIDEBAR)

- Dashboard
 - Revenue Analytics
 - Agents
 - Businesses
 - SMS
 - QRThrive Funnel
 - Expenses
 - Profit & Loss
 - Cash Flow
 - Settings
-

13. DESIGN PRINCIPLES

- Clean and minimal (no clutter)
 - Data-first layout
 - Mobile responsive (for quick checks)
 - Fast loading
 - Role-based visibility
-

14. FINANCIAL PLANNING & BUDGETING SYSTEM (CORE - NOT FUTURE)

This is now a REQUIRED core module, not a future feature.

A. BUDGET PLANNER (TARGET SETTING ENGINE)

Time Views

- Daily Budget
- Weekly Budget
- Monthly Budget
- Yearly Budget

Inputs (Editable by Admin)

- Target Revenue (e.g. ₦5,000,000/month)
- Target Businesses (e.g. 500 active businesses)
- Target SMS Usage
- Target Profit Margin (%)

Auto Calculations

- Required businesses to hit target
- Required revenue per day/week
- Required agent performance

Output View

- "You need 17 new businesses this week to stay on track"
 - "Each agent must bring 3 businesses this week"
-

B. FORECASTING ENGINE (VERY CRITICAL)

Forecast Types

- Revenue Forecast (next 3, 6, 12 months)
- Profit Forecast
- Cash Flow Forecast

Inputs

- Growth rate (%)
- Churn rate (%)
- Conversion rate (QRThrive → Vemtap)

Output

- Future MRR projection
 - Expected profit timeline
 - Risk alerts (e.g. decline trends)
-

C. BREAK-EVEN TRACKER

- Current revenue vs break-even point
 - Remaining gap to profitability
 - Estimated time to break-even
-

D. CASH FLOW PLANNER

View

- Daily / Weekly / Monthly cash tracking

Inputs

- Expected inflows
- Planned expenses

Output

- "You will run out of cash in X days if nothing changes"
 - "You have enough runway for X months"
-

E. SCENARIO SIMULATOR (HIGH VALUE FEATURE)

Scenarios

- Best Case
- Expected Case
- Worst Case

Adjustable Variables

- Pricing
- Growth rate
- Agent performance
- Churn rate

Output

- Profit comparison across scenarios
 - Risk exposure
-

F. GOAL TRACKING SYSTEM

- Monthly targets vs actual performance
- Weekly progress tracker
- Daily activity requirement

Visual Indicators:

- Green = On track
 - Yellow = At risk
 - Red = Off track
-

G. DECISION SUPPORT PANEL

System automatically tells you:

- "Increase agent recruitment"
 - "Reduce commission % to improve margin"
 - "Focus on Gold plan upgrades"
-

15. UPDATED NAVIGATION STRUCTURE (SIDEBAR)

- Dashboard
 - Financial Planning
 - Forecasting
 - Revenue Analytics
 - Agents
 - Businesses
 - SMS
 - QRThrive Funnel
 - Expenses
 - Profit & Loss
 - Cash Flow
 - Settings
-

This dashboard is your financial brain. It should not just show data — it should tell you what to do next, what targets to hit, and how to reach them.

16. DATABASE SCHEMA DESIGN (CORE FINANCIAL ENGINE)

This defines how all financial data is stored and connected.

A. AGENTS TABLE

Purpose: Track all agents and their performance

Fields:

- id (PK)
- name

- email
 - phone
 - status (active/inactive)
 - date_joined
 - total_businesses_acquired
 - total_revenue_generated
 - total_commission_earned
 - created_at
 - updated_at
-

B. BUSINESSES TABLE (Linked to Agents)

Fields:

- id (PK)
 - name
 - plan (free/silver/gold/platinum/enterprise)
 - monthly_payment
 - agent_id (FK → agents.id)
 - status (active/inactive)
 - join_date
 - last_payment_date
 - created_at
 - updated_at
-

C. TRANSACTIONS TABLE

Purpose: Record every money movement (core financial ledger)

Fields:

- id (PK)
 - type (subscription/sms/commission/expense)
 - platform (vemtap/qrthrive)
 - business_id (FK)
 - agent_id (FK, nullable)
 - amount
 - cost (if applicable)
 - profit
 - payment_method (card/bank/etc)
 - reference_id
 - date
 - created_at
-

D. BUDGETS TABLE

Purpose: Store targets and planned financial goals

Fields:

- id (PK)
 - period_type (daily/weekly/monthly/yearly)
 - target_revenue
 - target_businesses
 - target_sms_usage
 - target_profit
 - start_date
 - end_date
 - created_by
 - created_at
-

E. FORECASTS TABLE

Purpose: Store projected financial outcomes

Fields:

- id (PK)
 - forecast_type (revenue/profit/cashflow)
 - projected_value
 - growth_rate
 - churn_rate
 - conversion_rate
 - period (month/year)
 - scenario (best/expected/worst)
 - created_at
-

F. CASH_FLOW TABLE

Purpose: Track inflow and outflow of cash

Fields:

- id (PK)
- type (inflow/outflow)
- category (subscription/sms/salary/hosting/commission/etc)
- amount

- reference_id (optional link to transaction)
 - date
 - created_at
-

G. EXPENSES TABLE

Purpose: Detailed breakdown of costs

Fields:

- id (PK)
 - category (salary/hosting/tools/misc)
 - amount
 - frequency (one-time/recurring)
 - date
 - created_at
-

H. SMS_USAGE TABLE

Purpose: Track SMS activity and profitability

Fields:

- id (PK)
 - business_id (FK)
 - sms_count
 - cost_per_sms
 - selling_price_per_sms
 - total_cost
 - total_revenue
 - total_profit
 - date
-

I. QRTHRIVE_FUNNEL TABLE

Purpose: Track conversion funnel

Fields:

- id (PK)
- qr_scans
- leads_captured

- qr_users
 - converted_to_vemtap
 - conversion_rate
 - date
-

J. METRICS SNAPSHOT TABLE (VERY IMPORTANT)

Purpose: Store daily computed metrics for fast dashboard loading

Fields:

- id (PK)
 - date
 - total_revenue
 - total_profit
 - total_businesses
 - active_agents
 - churn_rate
 - conversion_rate
 - created_at
-

RELATIONSHIPS (HOW EVERYTHING CONNECTS)

- agents → businesses (1 to many)
 - businesses → transactions (1 to many)
 - transactions → cash_flow (linked via reference_id)
 - businesses → sms_usage (1 to many)
-

KEY DESIGN PRINCIPLES

- Keep transactions as single source of truth
 - Avoid duplicating financial data unnecessarily
 - Use snapshot tables for performance
 - Always track date for every record (for reporting)
-

This schema is the backbone of your financial system. If structured correctly, every dashboard, forecast, and insight will run smoothly on top of it.

17. CALCULATION ENGINE DESIGN (LOGIC LAYER)

This defines how raw data becomes financial intelligence. All calculations should be implemented as services/functions that read from the database and output metrics used across the dashboard.

A. CORE DEFINITIONS (SOURCE OF TRUTH)

- Gross Revenue = $\text{SUM}(\text{transactions.amount WHERE type IN ['subscription','sms','enterprise']})$
 - Total Costs = $\text{SUM}(\text{transactions.cost}) + \text{SUM}(\text{expenses.amount}) + \text{SUM}(\text{commission payouts})$
 - Net Profit = Gross Revenue - Total Costs
 - MRR (Monthly Recurring Revenue) = $\text{SUM}(\text{monthly_payment FROM active businesses})$
-

B. COMMISSION CALCULATION

For each transaction (subscription):

- $\text{commission} = \text{amount} * \text{commission_rate}$ (e.g. 30%)
- Store in $\text{transactions.profit} = \text{amount} - \text{commission} - \text{cost}$

Agent Earnings:

- $\text{agent_total_commission} = \text{SUM}(\text{commission WHERE agent_id} = X)$
-

C. SMS PROFIT ENGINE

Per record in `sms_usage`:

- $\text{total_cost} = \text{sms_count} * \text{cost_per_sms}$
- $\text{total_revenue} = \text{sms_count} * \text{selling_price_per_sms}$
- $\text{total_profit} = \text{total_revenue} - \text{total_cost}$

Aggregate:

- $\text{sms_total_profit} = \text{SUM}(\text{total_profit})$
-

D. DAILY / WEEKLY / MONTHLY AGGREGATIONS

For any period (`start_date` → `end_date`):

- $\text{revenue} = \text{SUM}(\text{transactions.amount})$

- $\text{profit} = \text{SUM}(\text{transactions.profit})$
- $\text{new_businesses} = \text{COUNT}(\text{businesses WHERE join_date in range})$

These power all charts and reports.

E. CHURN CALCULATION

- $\text{churn_rate} = (\text{lost_customers} / \text{total_customers_start_period}) * 100$

Where:

- $\text{lost_customers} = \text{businesses that moved from active} \rightarrow \text{inactive in period}$
-

F. CONVERSION ENGINE (QRTHRIVE $\rightarrow$ VEMTAP)

- $\text{conversion_rate} = (\text{converted_to_vemtap} / \text{qr_users}) * 100$
 - $\text{revenue_from_conversions} = \text{SUM}(\text{monthly_payment of converted users})$
-

G. BUDGET VS ACTUAL ENGINE

For each budget:

- $\text{actual_revenue} = \text{SUM}(\text{transactions.amount within budget period})$
- $\text{performance_ratio} = \text{actual_revenue} / \text{target_revenue}$

Output:

- If $\geq 1 \rightarrow$ On Track
 - If $0.7-0.99 \rightarrow$ At Risk
 - If $< 0.7 \rightarrow$ Off Track
-

H. TARGET BREAKDOWN ENGINE

Given:

- $\text{monthly_target_revenue}$

Compute:

- $\text{daily_target} = \text{monthly_target} / 30$
- $\text{weekly_target} = \text{monthly_target} / 4$

Required businesses:

- $\text{required_businesses} = \text{target_revenue} / \text{ARPU}$

Agent requirement:

- $\text{businesses_per_agent} = \text{required_businesses} / \text{total_active_agents}$
-

I. FORECAST ENGINE

For each future period:

- $\text{projected_businesses} = \text{current_businesses} * (1 + \text{growth_rate} - \text{churn_rate})$
- $\text{projected_revenue} = \text{projected_businesses} * \text{ARPU}$
- $\text{projected_profit} = \text{projected_revenue} * (1 - \text{commission_rate}) - \text{fixed_costs}$

Run this iteratively for 3, 6, 12 months.

J. CASH FLOW ENGINE

For a given period:

- $\text{inflow} = \text{SUM}(\text{cash_flow.amount} \text{ WHERE type} = \text{'inflow'})$
- $\text{outflow} = \text{SUM}(\text{cash_flow.amount} \text{ WHERE type} = \text{'outflow'})$
- $\text{net_cash} = \text{inflow} - \text{outflow}$

Runway:

- $\text{runway_months} = \text{current_cash_balance} / \text{monthly_burn}$
-

K. BREAK-EVEN ENGINE

- $\text{break_even_revenue} = \text{fixed_costs} / (1 - \text{commission_rate})$
 - $\text{gap_to_break_even} = \text{break_even_revenue} - \text{current_revenue}$
-

L. SCENARIO SIMULATION ENGINE

Inputs:

- growth_rate
- churn_rate
- commission_rate
- pricing (ARPU)

Run forecast engine with different inputs:

- Best Case (high growth, low churn)
- Expected Case
- Worst Case (low growth, high churn)

Output:

- Profit comparison table
 - Risk indicators
-

M. ALERT & DECISION ENGINE

Trigger rules:

- If churn_rate > 10% → Alert
- If profit < 0 → Alert
- If revenue growth < target → Alert

Decision outputs:

- "Increase agent acquisition"
 - "Push plan upgrades"
 - "Reduce cost"
-

N. SNAPSHOT GENERATION (PERFORMANCE)

Daily cron job:

- Calculate all key metrics
- Store in metrics_snapshot table

This ensures fast dashboard loading and historical tracking.

DESIGN PRINCIPLES

- All calculations must be reproducible from raw data
 - Avoid hardcoding values (use assumptions/config table)
 - Separate calculation logic from UI
 - Use scheduled jobs for heavy computations
-

This calculation engine transforms your database into a decision-making system. Without it, the dashboard is just visuals. With it, the system becomes your financial brain.

18. USER FLOW (DAILY, WEEKLY, MONTHLY OPERATIONS)

This defines how you (founder/admin) will actually use the system to run Vemtap.

A. DAILY OPERATIONS (10–15 MINUTES MAX)

Step 1: Open Dashboard (Morning Check)

Review:

- Today's Revenue vs Daily Target
- New Businesses Today
- SMS Revenue Today
- Cash Position

Step 2: Check Alerts Panel

Look for:

- Revenue drop
- Low agent activity
- High churn warning

Step 3: Action Decisions

System tells you:

- "You need X more businesses today"

Your actions:

- Push agents (WhatsApp / call)
- Follow up on pending businesses
- Trigger promotions if needed

Step 4: Agent Quick Check

- Who added businesses today?
- Who added zero?

Action:

- Nudge inactive agents immediately
-

B. WEEKLY OPERATIONS (DEEP CONTROL)

Step 1: Review Weekly Performance

- Weekly Revenue vs Target
- Businesses acquired this week
- Top vs worst agents

Step 2: Agent Review

- Rank agents by performance
- Identify underperformers

Actions:

- Reward top agents
- Train or replace weak agents

Step 3: Funnel Review (QRThrive)

- QR scans → conversions

Action:

- Improve conversion (better onboarding, offers)

Step 4: Expense Review

- Check commissions paid
 - Check unusual costs
-

C. MONTHLY OPERATIONS (STRATEGIC LEVEL)

Step 1: Full P&L Review

- Total Revenue
- Total Cost
- Net Profit

Step 2: Growth Analysis

- Business growth rate
- Churn rate
- Plan upgrades/downgrades

Step 3: Forecast Review

- Next 3–6 month projection

Ask:

- Are we growing fast enough?

Step 4: Budget Reset

- Set new targets:
 - Revenue
 - Businesses
 - Profit

System recalculates:

- Daily targets
- Weekly targets
- Agent expectations

D. MONTH-END DECISION POINTS

You must decide:

- Increase pricing?
 - Increase/decrease commission?
 - Hire more agents?
 - Invest more in growth?
-

E. YEARLY OPERATIONS (BIG STRATEGY)

- Total revenue for the year
- Total profit
- Total businesses acquired

Decisions:

- Expand to new locations
 - Raise funding or bootstrap
 - Launch new features/products
-

F. OPERATIONAL PRINCIPLE

Every check must lead to an action.

If the system shows:

- Data → You decide
 - Problem → You act
 - Opportunity → You scale
-

G. SIMPLE RULE

- Daily = Control
 - Weekly = Optimize
 - Monthly = Strategize
 - Yearly = Expand
-

This flow ensures you are not just watching your business — you are actively driving it every single day.

19. DEVELOPER-READY API & BACKEND ARCHITECTURE

This section translates the system into actual backend implementation structure.

A. ARCHITECTURE OVERVIEW

Recommended Stack (flexible):

- Backend: Node.js (Express) or Laravel or Django
- Database: PostgreSQL (preferred) or MySQL
- Cache: Redis (for fast metrics)
- Jobs/Queues: Bull (Node) / Celery (Python) / Laravel Queue

Architecture Style:

- Service-based (modular)
 - REST API (or GraphQL optional)
-

B. CORE SERVICES (VERY IMPORTANT)

Split logic into services:

1. Auth Service
2. Agent Service
3. Business Service
4. Transaction Service
5. Financial Engine Service
6. Forecast Service
7. Budget Service
8. SMS Service
9. Analytics Service

Each service handles its own logic and communicates via API.

C. API ENDPOINTS DESIGN

1. AUTH

- POST /api/auth/login
 - POST /api/auth/register
 - GET /api/auth/me
-

2. AGENTS

- GET /api/agents
- POST /api/agents
- GET /api/agents/{id}

- PUT /api/agents/{id}
 - GET /api/agents/{id}/performance
-

3. BUSINESSES

- GET /api/businesses
 - POST /api/businesses
 - GET /api/businesses/{id}
 - PUT /api/businesses/{id}
-

4. TRANSACTIONS (CORE LEDGER)

- GET /api/transactions
 - POST /api/transactions
 - GET /api/transactions/{id}
-

5. FINANCIAL METRICS

- GET /api/metrics/dashboard

Returns:

- MRR
 - Revenue
 - Profit
 - Active businesses
-

6. BUDGETS

- GET /api/budgets
 - POST /api/budgets
 - GET /api/budgets/{id}
-

7. FORECAST

- GET /api/forecast

Query params:

- period=3 | 6 | 12 months
 - scenario=best | expected | worst
-

8. CASH FLOW

- GET /api/cashflow
 - POST /api/cashflow
-

9. SMS

- GET /api/sms/usage
 - POST /api/sms/send
-

10. QRTHRIVE FUNNEL

- GET /api/funnel
-

D. FINANCIAL ENGINE SERVICE (CORE LOGIC)

This is NOT exposed directly.

Responsibilities:

- Calculate profit
- Calculate commissions
- Run forecasts
- Compute KPIs

Example Functions:

- calculateMRR()
 - calculateProfit(startDate, endDate)
 - calculateChurn()
 - runForecast(params)
-

E. BACKGROUND JOBS (CRITICAL)

Use queues/cron jobs for heavy tasks:

Daily Jobs

- Compute metrics snapshot
- Store in metrics_snapshot table

Weekly Jobs

- Agent performance aggregation

Monthly Jobs

- Generate P&L snapshot
 - Forecast recalculation
-

F. CACHING STRATEGY

Use Redis to cache:

- Dashboard metrics
- Frequently requested reports

Cache expiry:

- 5–15 minutes for dashboard
-

G. DATA FLOW (HOW SYSTEM WORKS)

1. Business pays subscription
 2. Transaction is created
 3. Financial Engine processes:
 4. commission
 5. profit
 6. Data stored in transactions
 7. Metrics updated via background job
 8. Dashboard fetches from metrics API
-

H. SECURITY & ACCESS CONTROL

Roles:

- Super Admin
- Finance Admin
- Manager (limited)

Controls:

- JWT authentication
 - Role-based access middleware
-

I. SCALABILITY CONSIDERATIONS

- Separate read/write queries (optional later)
 - Use pagination for all lists
 - Index key fields (date, agent_id, business_id)
-

J. DEVELOPMENT PRINCIPLES

- Keep business logic in services (not controllers)
 - Never calculate financial data in frontend
 - Always log transactions (no silent changes)
 - Use environment config for rates (commission, SMS cost)
-

This architecture ensures your system is scalable, maintainable, and accurate. It allows your financial engine to run reliably as Vemtap grows.